Decentralized Electoral Architectures: A Comprehensive Specification for Phase 1 Blockchain-Based E-Voting

The Evolution of Trust in Digital Democratic Systems

The integrity of democratic systems rests fundamentally upon the security, privacy, and transparency of voting procedures. Traditional electoral mechanisms, ranging from physical paper ballots to centralized electronic voting machines, have increasingly faced challenges that threaten public confidence, including voter coercion, identity theft, and the lack of independent auditability.[1] As technology advances, the shift toward decentralized architectures—specifically those leveraging blockchain technology—offers a transformative potential to modernize electoral systems.[3] By providing an immutable, tamper-proof record of transactions, blockchain ensures that once a vote is cast, it remains a permanent part of the ledger, beyond the manipulation of intermediaries or external bad actors.[3]

In the context of this college project, the transition to a blockchain-enabled e-voting system is designed to solve systemic issues such as manual errors in counting, lack of real-time results, and the opacity of traditional tallies.[6] For Phase 1, the scope is intentionally focused on the core interactions between two primary personas: the Admin and the Voter.[7] This foundational phase prioritizes the establishment of secure identity anchors—utilizing national biometric standards like Aadhaar—and the differentiation between institutional "Private" elections and jurisdictional "Public" elections.[7] By anchoring digital identities in verifiable physical data, the system mitigates the "Sybil attack" risks inherent in decentralized systems where a single actor might otherwise create multiple identities to subvert the outcome.[4]

The conceptual framework of this system relies on the Ethereum network, where smart contracts written in Solidity govern the logic of candidate registration, voter eligibility, and the mathematical verification of the tally.[9] This report details the user flows, technical requirements, and cryptographic mechanisms necessary to build a robust Phase 1 prototype that balances the transparency of a public ledger with the stringent privacy requirements of a secret ballot.[11]

Fundamental Parameters of the Phase 1 Environment

For the initial implementation, several architectural assumptions have been made to ensure the stability of the core flows while maintaining the potential for future scalability. The platform is built upon a multi-tenant architecture, allowing different organizations (colleges or

governmental bodies) to host distinct electoral events within the same framework.[7]

Core Role Definitions and Access Control

Role	Responsibility	Initialization Method	Primary Goal
Admin	Configuration, Candidate Management, Eligibility Locking, and Deployment.[7]	Pre-created by Platform/Super Admin.[7]	Facilitate a secure, immutable election event.
Voter	Self-Registration, Pre-Verification, and Secure Ballot Casting.[7]	Self-Signup with Aadhaar/ID Verification.[7]	Exercise suffrage with guaranteed privacy.

Admins in Phase 1 do not participate in a public signup process; instead, their accounts are provisioned by the platform to maintain a closed-loop of organizational authority.[7] This prevents unauthorized entities from creating fraudulent elections. In contrast, voters are granted a path to self-registration, provided they can verify their identity through the Aadhaar-linked OTP (One-Time Password) mechanism.[7] This dual-approach ensures that while the management of elections is centralized within authorized entities, participation remains inclusive and accessible to the verified population.[7]

Voter Life Cycle: Registration and Identity Anchoring

The voter's journey begins with the establishment of a verified digital identity that bridges their physical persona with the blockchain environment. This process is critical for ensuring the "one person, one vote" principle.[9]

New Flow VO: Voter Signup and Profile Creation

The signup process is the primary gatekeeper of the system's integrity. It collects the necessary identifiers to map a voter to both institutional (Private) and governmental (Public) election scopes.[7]

- **User and Goal**: A new voter (citizen, student, or employee) seeking to establish a persistent profile that allows them to participate in relevant elections within their organization or geographic region.[7]
- **Entry Point**: The user accesses the platform's landing page and selects the "Sign Up"

option.[7]

- **Sequence of Steps**:
  1. The system presents the Voter Registration Form, requiring a comprehensive set of personal and identification data.[7]
  2. The user enters their primary contact details (Email and Phone Number) and their 12-digit Aadhaar number, which serves as the biometric anchor for their identity.[7]
  3. The user provides their Full Name and a detailed Residential Address, including locality, city, state, and PIN code.[7]
  4. Optional fields for Student ID and Employee ID are provided to allow the system to map the user to institutional databases for private elections.[7]
  5. Upon submission, the backend performs a validation check to ensure that the Aadhaar number and contact details are not already associated with an existing account.[7]
  6. The system triggers an OTP to the user's phone number (linked to their Aadhaar) using an authentication API such as Firebase PhoneAuth.[9]
  7. The user verifies their identity by entering the received OTP.[7]
  8. On success, the system generates a Voter profile, issues a session JWT (JSON Web Token), and redirects the user to the Voter Dashboard.[7]
- **Decision Points**:
  - **Data Integrity**: Are the mandatory fields valid and the Aadhaar number unique in the system? If not, the system returns specific validation errors.[7]
  - **OTP Verification**: Does the provided OTP match the one generated by the authentication service? If it fails, the user is given options to retry or resend.[7]
- **Endpoint**: A verified voter account exists with a complete profile (IDs and address) stored in the database, and the user is logged into the dashboard.[7]

Administrative Control: Election Configuration and Scope

The Admin persona is tasked with the lifecycle management of an election, from the drafting of parameters to the final deployment of the smart contract on the blockchain.[7]

A1. Admin Login and Dashboard Access

- **User and Goal**: A pre-created Organization or Government Admin seeking to manage elections for their specific jurisdiction.[7]
- **Entry Point**: Common login page for the platform.[7]
- **Sequence of Steps**:
  1. The admin enters their email and password.[7]
  2. The system validates the credentials and confirms the "Admin" role status.[7]
  3. The admin is redirected to their Dashboard, which displays a filtered list of elections

belonging to their organization.[7]

- **Decision Points**:
  - **Authorization**: Does the user have the "Admin" role and is their status "ACTIVE"? If suspended or denied, access is blocked.[7]
- **Endpoint**: The admin is on the Dashboard with the ability to create or manage existing elections.[7]

A2. Strategic Election Creation: Private vs. Public Modalities

A central innovation of this project is the differentiation between Private and Public elections, which require distinct logic for eligibility and verification.[7]

- **User and Goal**: An Admin aiming to create a new draft election with specific visibility and timing parameters.[7]
- **Entry Point**: "Create New Election" button on the Admin Dashboard.[7]
- **Sequence of Steps**:
  1. The system opens the Create Election form.[7]
  2. The admin defines the Metadata: Election title, description, and election type (single-seat or multi-seat).[7]
  3. The admin selects the Visibility Type: **Private** or **Public**.[7]
  4. The admin defines the Timings: Start and end times for the voting window.[7]
  5. The admin defines the Scope: For a college (Private), this might be a specific department. For a government (Public), this might be a municipal ward.[7]
  6. The backend validates that the timings are in the future and that the admin is authorized for the chosen visibility type.[7]
- **Decision Points**:
  - **Temporal Logic**: Is the start time before the end time and in the future?.[7]
  - **Visibility Authorization**: Is the admin permitted to create a Public election (government level) or only a Private one (college level)?.[7]
- **Endpoint**: A DRAFT election is created with the specified VisibilityType stored in the database.[7]

Detailed Differentiation of Election Types

Feature	Private Election (Institutional)	Public Election (Governmental)
Organization	Colleges, Universities, Corporate Bodies.[7]	Municipalities, State/National Governments.[7]

Eligibility Data	Uploaded CSV of Student/Employee IDs.[7]	Geographical boundaries (PIN, Ward, District).[7]
Verification Basis	Matching a unique ID against a Merkle Root.[7]	Geospatial mapping of physical address.[7]
Primary Use Case	Student Council, Internal board elections.[9]	Local ward or municipal representative elections.[7]

Eligibility Engineering: Cryptographic and Geospatial Methods

The mechanism for determining who can cast a ballot is the most complex component of the Admin flow, bifurcated by election type.[7]

A4-P. Configure Eligibility for Private Elections (ID Lists)

Private elections rely on a predefined universe of voters. To maintain efficiency and privacy on the blockchain, the system utilizes a Merkle Tree structure.[18]

- **User and Goal**: An Admin configuring a college election who needs to restrict voting to a specific list of students.[7]
- **Entry Point**: "Manage Voters / Eligibility (Private)" section of the configuration page.[7]
- **Sequence of Steps**:
  1. The admin uploads a CSV file containing the unique IDs (e.g., USNs or Employee IDs) of eligible voters.[7]
  2. The system parses the list, checking for formatting errors and duplicates.[7]
  3. The backend stores these IDs in an electionvoters table with an AUTHORIZED status.[7]
  4. The admin reviews the total count of eligible voters.[7]
  5. The admin clicks "Lock Eligibility."
  6. The system generates a Merkle Root: it hashes each ID, pairs them, and hashes the pairs recursively until a single root hash remains.[18]
  7. The Merkle Root is stored as the "Ground Truth" for the smart contract.[18]
- **Decision Points**:
  - **Validation**: Are the uploaded IDs unique and valid for the institution?.[7]
  - **State Locking**: Is there at least one voter before locking?.[7]
- **Endpoint**: The Private election has a locked eligibility list represented by a Merkle Root.[7]

The Logic of Merkle Trees in E-Voting

A Merkle tree is a fundamental component of blockchain technology that allows for efficient and secure verification of large datasets.[20] In Phase 1, the system uses the Merkle Root to represent the entire eligible voter list. When a voter attempts to cast their ballot, they must

provide a "Merkle Proof"—a small set of hashes that, when combined with their own ID hash, recreates the Merkle Root stored in the smart contract.[18] This avoids the need to store thousands of student IDs directly on the blockchain, which would be prohibitively expensive in terms of gas fees.[18]

A4-U. Configure Eligibility for Public Elections (Location-Based)

Public elections do not utilize a fixed list of IDs; instead, they are open to any voter whose registered address falls within the specified geographic boundaries.[7]

- **User and Goal**: An Admin configuring a governmental election seeking to define a geographic constituency.[7]
- **Entry Point**: "Configure Eligibility (Public)" on the election configuration page.[7]
- **Sequence of Steps**:
  1. The admin selects the geographical units (e.g., specific Districts, Wards, or ranges of PIN codes).[7]
  2. The system saves these inclusion rules.[7]
  3. The backend performs a "dry-run" query to estimate the number of registered voters whose addresses currently match these criteria.[7]
  4. The admin reviews the estimate and confirms the rules.[7]
  5. The admin clicks "Lock Location-Based Eligibility".[7]
  6. The system generates a hash of the rule set to ensure that boundaries cannot be altered once the election is live.[7]
- **Decision Points**:
  - **Geographic Integrity**: Are the selected wards or PIN codes valid and contiguous if required by policy?.[7]
- **Endpoint**: The Public election has locked location-based eligibility rules.[7]

Geospatial Mapping and Geocoding Logic

For public elections, the system must bridge the gap between a physical address (e.g., "123 Main St, Bangalore") and a digital boundary.[21] Geocoding is the computational process of converting a physical street address into geographic coordinates (latitude and longitude).[22] Once coordinates are established, the system uses a spatial query to determine which electoral ward or constituency "polygon" contains that point.[23] In Phase 1, this mapping happens during the "Pre-Verification" flow for the voter, ensuring that their profile data is cross-referenced with the election's locked rules.[7]

Candidate Management and Deployment

The final phase for the Admin involves populating the ballot and committing the election to the blockchain network.[7]

A3. Add and Finalize Candidate List

- **User and Goal**: Admin adding the contesting candidates to the draft election.[7]
- **Entry Point**: "Manage Candidates" on the configuration page.[7]
- **Sequence of Steps**:
  1. The admin fills in candidate details: Name, Unique Identifier (USN or ID), Party/Organization name, Symbol/Logo, and Manifesto.[7]
  2. The backend checks that the candidate belongs to the correct institutional or geographic scope.[7]
  3. The admin saves the candidate; the system validates that no duplicate candidate exists for this election.[7]
  4. Once all candidates are added, the admin clicks "Finalize Candidate List".[7]
  5. The system sorts the candidates, generates a Candidate List Hash, and locks the list.[7]
- **Decision Points**:
  - **Candidate Validation**: Is the candidate eligible to run based on their own profile status?.[7]
  - **Finalization**: Is there at least one candidate before locking the ballot?.[7]
- **Endpoint**: The candidate list is locked and immutable.[7]

A5. Finalize Election and Deploy Smart Contract

This flow moves the election from the backend database to the immutable blockchain.[7]

- **User and Goal**: Admin reviewing the final configuration and triggering the blockchain deployment.[7]
- **Entry Point**: "Review & Finalize Election" summary screen.[7]
- **Sequence of Steps**:
  1. The system displays a summary of election metadata, the candidate list, and the eligibility status (Merkle Root or Location Hash).[7]
  2. The admin clicks "Finalize & Deploy".[7]
  3. The backend calls a Blockchain Interaction Service to deploy the Smart Contract.[9]
  4. The contract is initialized with the start time, end time, candidate identifiers, and the eligibility proof (Merkle Root or location hash).[7]
  5. The blockchain returns a unique Contract Address.[7]
  6. The backend stores the contract address and updates the election status to SCHEDULED or ACTIVE.[7]
- **Decision Points**:
  - **Deployment Success**: If the smart contract fails to deploy (e.g., due to network congestion or insufficient gas), the admin is given an option to retry without altering the locked parameters.[7]
- **Endpoint**: The election is live on the blockchain.[7]

Voter Experience: Dashboard and Pre-Verification

Once an election is live, the Voter's interactions are focused on confirming their ability to vote and then securely casting their ballot.[7]

V1. Voter Login and Dashboard

- **User and Goal**: A registered voter logging in to see available elections.[7]
- **Sequence of Steps**:
  1. The voter logs in using their credentials.[7]
  2. The system identifies the user's role and redirects them to the Voter Dashboard.[7]
  3. The dashboard displays three categories: Upcoming, Running (Active), and Completed elections.[7]
  4. Each election card indicates its visibility (Private/Public) and the voter's verification status.[7]
- **Endpoint**: The voter has clear visibility of their electoral opportunities.[7]

V2'. One-Time Pre-Verification

To minimize friction on voting day and manage blockchain transaction load, Phase 1 includes a pre-verification step.[7]

- **User and Goal**: A voter confirming they are eligible for a specific election before the window opens.[7]
- **Entry Point**: Voter clicks an election card on their dashboard.[7]
- **Sequence of Steps**:
  1. The Election Details page shows the requirements for that specific election.[7]
  2. **For Private Elections**: The voter enters their Student/Employee ID. The system checks if this ID matches the AUTHORIZED list for that election and the ID stored in the voter's own profile.[7]
  3. **For Public Elections**: The system displays the voter's registered address. The voter clicks "Verify Eligibility." The backend maps their address to the election's locked geography rules.[7]
  4. If the mapping/matching is successful, the system creates a verification record in the database.[7]
  5. The election status on the voter's dashboard updates to "Verified for this election".[7]
- **Decision Points**:
  - **Eligibility Verification**: Does the ID match? Does the address fall within the ward? If not, the system explains why they are not eligible.[7]
- **Endpoint**: The voter is cleared to vote when the election becomes ACTIVE.[7]

The Act of Suffrage: Secure Vote Casting

The core value proposition of the system is the secure, tamper-proof casting of the ballot on the blockchain.[5]

V3'. Secure Vote Casting in an Active Election

- **User and Goal**: A verified voter casting their single allowed vote for their chosen candidate.[7]
- **Entry Point**: A "Running" election on the Dashboard marked "Verified".[7]
- **Sequence of Steps**:
  1. The voter clicks "Vote".[7]
  2. The backend performs the "Pre-Ballot Check":
     - Is the election currently ACTIVE (within start/end times)?.[7]
     - Is the voter VERIFIED for this specific election?.[7]
     - Has the voter already voted (checked via an on-chain mapping of address-to-election)?.[7]
  3. If checks pass, the Ballot Screen is displayed with candidate names and symbols.[7]
  4. The voter selects a candidate and confirms their choice in a modal dialog.[7]
  5. The system triggers the blockchain transaction.[7]
  6. The smart contract logic:
     - Validates the time and voter's has-voted status.[9]
     - For Private: Verifies the Merkle Proof against the Root.[18]
     - Records the vote for the candidate and marks the voter as "Voted" on the immutable ledger.[7]
  7. On success, the system stores the Transaction Hash and displays a success screen.[6]
- **Decision Points**:
  - **Double-Voting Prevention**: Does the blockchain return an error indicating the voter has already participated? The system must ensure that no account can successfully trigger the vote() function twice for the same contract.[5]
- **Endpoint**: One valid vote is recorded on-chain; the voter is barred from further attempts.[7]

Transactional Logic and Gas Optimization

The cost of casting a vote on the Ethereum network (gas) is a significant factor for college projects.[12] By using Merkle Trees for verification and off-chain pre-verification for geographic rules, Phase 1 keeps on-chain computation to a minimum. The gas required is primarily for:

1. Verifying the $O(\log n)$ Merkle Proof path.[18]
2. Incrementing the candidate's vote counter ($SSTORE$ operation).[18]

3. Updating the voter's participation status to prevent double-voting.[10]

Example Gas Cost Estimates for Voting Operations

Operation	Complexity	Gas Consumption (Estimated)	Frequency
Deploy Election Contract	High	1,500,000 - 3,000,000	Once per election.
Cast Vote (Standard)	Low	40,000 - 80,000	Once per voter.
Cast Vote (with Merkle Proof)	Medium	80,000 - 150,000	Once per voter in Private elections.
Finalize/End Election	Low	30,000 - 60,000	Once per election.

Transparency, Tallying, and Trust

The conclusion of an election shifts the focus from participation to verifiability.[5]

V4. View Final Results and Transparency

- **User and Goal**: Any logged-in user seeking to view the outcome and verify its legitimacy.[7]
- **Entry Point**: "Completed Elections" section on the dashboard.[7]
- **Sequence of Steps**:
  1. The user selects a completed election where the status is "Published".[7]
  2. The Results Page displays the final tally for each candidate and identifies the winner.[7]
  3. The system provides "Transparency Proofs":
     - Link to the Smart Contract on a block explorer (e.g., Etherscan).[7]
     - The transaction hash of the "Finalize" call which closed the voting.[7]
     - Option to view individual (anonymized) transaction hashes for auditing.[20]
- **Decision Points**:
  - **Verification Status**: If the admin has not yet audited and published results, the page displays "Results Pending".[7]
- **Endpoint**: The user is satisfied that the results are consistent with the blockchain record.[7]

Cryptographic Integrity and Auditability

The use of blockchain provides "end-to-end verifiability." This means that anyone can verify that:

1. **Voter Integrity**: Only eligible voters were allowed to participate (via Merkle Root verification).[18]
2. **Cast-as-Intended**: The voter's ballot was recorded exactly as they selected it (verified via their personal transaction hash).[6]
3. **Counted-as-Cast**: All recorded ballots were included in the final tally (verified by summing the transactions on the public ledger).[11]

Security Infrastructure and Privacy Preservation

Phase 1 focuses on foundational security, ensuring that while the process is transparent, the voter's privacy is maintained to the extent possible within a public ledger framework.[11]

Multi-Factor Authentication (MFA) and Session Security

The platform employs a tiered security approach to protect both the admin's management capabilities and the voter's account.[27]

Security Layer	Implementation Mechanism	Role Protected
Biometric Anchor	Aadhaar Number uniqueness.[13]	Voter (Identity theft prevention).
Possession Factor	Aadhaar-linked OTP (Firebase PhoneAuth).[9]	Voter (Account signup).
Session Control	JWT (JSON Web Tokens) with short expiry.[7]	Both (Login security).
On-Chain Identity	Wallet/Account signature for every vote.[9]	Voter (Non-repudiation).
Administrative Locking	Digital signatures on Merkle Roots/ID lists.[7]	Admin (Tamper-proof configuration).

Addressing the Secrecy of the Ballot

One of the inherent tensions in blockchain voting is the transparency of the ledger versus the secrecy of the ballot.[8] In a standard public blockchain, the transaction link between an address

and a candidate is visible. For Phase 1, the system mitigates this by:

1. **Pseudonymity**: Only the blockchain address is recorded on-chain, not the voter's Aadhaar or student ID.[4]
2. **Hashing**: All sensitive identifiers are hashed before being used as keys in the smart contract mappings.[14]
3. **Future-Proofing**: The architecture is designed to eventually integrate Zero-Knowledge Proofs (ZKP), which allow a voter to prove they are eligible and have voted without revealing *how* they voted or *which* address they used.[1]

Technical Roadmap for Phase 1 Prototype Development

To finalize the scope of the project, the development team must adhere to a specific sequence of implementation tasks derived from the core flows.

Database Schema Considerations (Off-Chain)

While the voting happens on-chain, the management of profiles and election drafts requires a robust traditional database (e.g., PostgreSQL or MongoDB).[7]

1. **Users Table**: Stores email, phone, hashed Aadhaar, address coordinates, and institutional IDs.[7]
2. **Elections Table**: Stores title, description, visibility type, start/end times, and the smart contract address.[7]
3. **ElectionVoters Table**: Maps voters to elections, storing their AUTHORIZED status and VOTED flag for the UI.[7]
4. **Candidates Table**: Stores candidate details linked to an election ID.[7]

Smart Contract Logic (On-Chain)

The Solidity contract for Phase 1 must implement several critical functions:

- constructor(...): Initializes the election with times, candidates, and the eligibility proof.[7]
- vote(uint candidateId, bytes32 merkleProof): The core voting function. It checks block.timestamp, verifies the merkleProof against the merkleRoot, ensures hasVoted[msg.sender] is false, and increments the candidate's tally.[10]
- getResults(): A read-only function that returns the current tally once the election has ended.[10]
- endElection(): An admin-only function to lock the contract and prevent further participation after the deadline.[9]

GIS Integration for Public Elections

The public election flow requires integration with a Geocoding API. The logic flow for verification is as follows:

$$\text{Address} \rightarrow \text{API (Geocodio/Google)} \rightarrow (Lat, Long)$$

$$(Lat, Long) + \text{Constituency Polygons} \rightarrow \text{Constituency ID}$$

$$\text{Constituency ID} == \text{Election Scope} ? \text{VERIFIED} : \text{REJECTED}$$

This mapping ensures that governmental eligibility is strictly enforced based on the voter's registered domicile.[19]

Conclusion and Strategic Outlook

The Phase 1 architecture described in this report provides a comprehensive foundation for a blockchain-based e-voting system suitable for a college project and beyond.[6] By focusing on two primary roles—Admin and Voter—and differentiating between Private institutional and Public jurisdictional elections, the system addresses the diverse needs of modern electoral environments.[7]

The use of Aadhaar-based identity anchoring solves the fundamental problem of digital personhood, while Merkle Trees and geospatial mapping provide scalable and secure mechanisms for eligibility verification.[13] The result is a system that not only ensures the integrity of the count but also builds voter trust through transparency and independent auditability.[3] As the project progresses beyond Phase 1, the integration of advanced cryptographic techniques like Zero-Knowledge Proofs and decentralized identity (DID) will further refine the balance between privacy and verifiability, paving the way for a more robust and inclusive digital democracy.[8]

TECH STACK

Backend—server - python flask

database-MYSQL

Frontend– framework- react , and HTML JS

cache–REDIS

Background workers– REDIS RQ

Works cited

1. Secure and privacy-preserving voting system using zero- knowledge proofs - Semantic Scholar, accessed February 18, 2026, https://pdfs.semanticscholar.org/9a87/9acae37d7cf39a7acbc8f4486c6d73bfdcba.pdf
2. (PDF) Secure and Privacy-Preserving Voting System Using Zero-Knowledge Proofs, accessed February 18, 2026, https://www.researchgate.net/publication/372823212_Secure_and_Privacy-Preserving_Voting_System_Using_Zero-Knowledge_Proofs
3. Blockchain-Based Voting Systems Enhancing Transparency and Security in Electoral Processes - ITM Web of Conferences, accessed February 18, 2026, https://www.itm-conferences.org/articles/itmconf/pdf/2025/07/itmconf_icsice2025_02004.pdf
4. Integrating Blockchain Technology for Privacy-Preserving and Tamper- Proof Electronic Voting in Modern Electoral Systems - Infonomics Society, accessed February 18, 2026, https://infonomics-society.org/wp-content/uploads/Integrating-Blockchain-Technology-for-Privacy-Preserving-and-Tamper-Proof-Electronic-Voting.pdf
5. Design and Development of a Blockchain-based Electronic Voting System - ResearchGate, accessed February 18, 2026, https://www.researchgate.net/publication/394242418_Design_and_Development_of_a_Blockchain-based_Electronic_Voting_System
6. Online Voting System For Our College Management Using Blockchain - IJSART, accessed February 18, 2026, https://ijsart.com/public/storage/paper/pdf/IJSARTV11I4103242.pdf
7. We want to work on college project, related e voti.pdf
8. Modernizing Voting Systems: A Comprehensive Approach Using Blockchain, Biometrics and Zero Knowledge Proofs, accessed February 18, 2026, https://ijecbe.ui.ac.id/go/article/download/116/83
9. College Election System using Blockchain - ITM Web of Conferences, accessed February 18, 2026, https://www.itm-conferences.org/articles/itmconf/pdf/2022/04/itmconf_icacc202

2_03005.pdf
10. (PDF) Blockchain-enabled Secured and Transparent E-Voting System using Smart Contracts - ResearchGate, accessed February 18, 2026, https://www.researchgate.net/publication/391851449_Blockchain-enabled_Secured_and_Transparent_E-Voting_System_using_Smart_Contracts
11. Blockchain-Enabled E-Voting for Secure and Transparent Elections - SciTePress, accessed February 18, 2026, https://www.scitepress.org/Papers/2025/138662/138662.pdf
12. Distributed Anonymous e-Voting Method Based on Smart Contract Authentication - MDPI, accessed February 18, 2026, https://www.mdpi.com/2079-9292/12/9/1968
13. Online Voting Using Aadhaar Card with Multi-Factor Verification, accessed February 18, 2026, https://www.ijset.in/wp-content/uploads/IJSET_V13_issue5_658.pdf
14. Secured Blockchain-Based Voting System Using ZKP, IPFS - IJRASET, accessed February 18, 2026, https://www.ijraset.com/research-paper/secured-blockchain-based-voting-system
15. How to Link Aadhaar Card to Voter ID? A Step-by-Step Guide - Aditya Birla Capital, accessed February 18, 2026, https://www.adityabirlacapital.com/abc-of-money/how-to-link-aadhaar-to-voter-id
16. Guidelines On Mobile as Digital Identity - e-Governance Standards, accessed February 18, 2026, https://egovstandards.gov.in/sites/default/files/2021-07/Guidelines%20on%20Mobile%20as%20Digital%20identity.pdf
17. Secure E-Voting System With Biometric Authentication - Atlantis Press, accessed February 18, 2026, https://www.atlantis-press.com/article/126017437.pdf
18. Scalable Open-Vote Network on Ethereum - Financial Cryptography 2020, accessed February 18, 2026, https://fc20.ifca.ai/wtsc/WTSC2020/WTSC20_paper_10.pdf
19. GIS & Elections, accessed February 18, 2026, https://www.eac.gov/sites/default/files/electionofficials/QuickStartGuides/GIS_Elections_EAC_Quick_Start_Guide_508.pdf
20. Voter Verification in an Election Using Merkle Tree - IJRASET, accessed February 18, 2026, https://www.ijraset.com/research-paper/voter-verification-in-an-election-using-merkle-tree
21. Map voter data to plan your campaign | Documentation - Learn ArcGIS, accessed February 18, 2026, https://learn.arcgis.com/en/projects/map-voter-data-to-plan-your-campaign/
22. Address Matching Voting Records to Map Coordinates, accessed February 18, 2026, https://data.wvgis.wvu.edu/pub/VTD/Resource/Geocode/Geocode_Match_Report_Example.pdf

23. GIS in Voting by Keet Consulting Services, LLC | Esri Partner Solution, accessed February 18, 2026, https://www.esri.com/partners/keet-consulting-serv-a2T70000000TNXCEA4/gis-in-voting-a2d70000000VJeWAAW
24. GIS and the Future of Voter Registration Systems - NASS.org, accessed February 18, 2026, https://www.nass.org/sites/default/files/2018-07/bpro-white-paper-nass-summer18.pdf
25. Geographic Base of a Voter Register —, accessed February 18, 2026, https://aceproject.org/main/english/et/etl02.htm
26. Zero-Knowledge Proof In Voting Systems - Meegle, accessed February 18, 2026, https://www.meegle.com/en_us/topics/zero-knowledge-proofs/zero-knowledge-proof-in-voting-systems
27. Design and Implementation of a Blockchain-Based E-Voting System with Enhanced Voter Authentication - ijarsct, accessed February 18, 2026, https://ijarsct.co.in/Paper25672.pdf
28. Enhancing the Security and Privacy of Online Voting Systems Using Zero-Knowledge Proofs in Blockchain - International Journal of Engineering Research & Technology, accessed February 18, 2026, https://www.ijert.org/enhancing-the-security-and-privacy-of-online-voting-systems-using-zero-knowledge-proofs-in-blockchain